fibre links and talking with each other using TCP/IP protocols.

Many-cores are a research project currently, much like the 10 GHz processor existed on paper not so long ago. We do have some multi-core solutions in the market though—ranging between dual-cores, quad-cores and looking forward to octa-cores. You could also use two dual-core processors on a single motherboard, for a pseudo-quad-core solution, or indeed two quad-cores for an eight-way powerhouse of data. The last example can be picked from Intel today—the Intel Desktop Board D5400XS, part of its "Extreme" range of motherboards, can house dual CPUs, providing up to eight core processing.

You also have more esoteric solutions in the making. Intel's looking to enter the discrete graphics processor market with its Larabee initiative (we spoke about this in the May 2008 issue of Digit). Larabee leverages multi-core capabilities to churn out graphic-intensive tasks. From the other side, AMD and NVIDIA are turning the GPU into a GP-GPU—a more General Purpose processor which can offload some tasks off the main processing unit(s).

Today, almost everyone has a multi-core chip powering his or her desktop. So why doesn't processing feel faster than it did when speeds reigned supreme? The main reason is that there is a lag between hardware that has gone multi-core, and software that is still largely stuck in the past.

Past Tense

Multi-core processors rely on the existence of software code that does things in parallel, using so-called 'multi-threaded' applications. Give one chunk of this task to this processor here, the other to that one, now quick, both of you run at 2.4 GHz and give me the result in half time—effectively making the multi-core 4.8GHz fast. Yes, that was a bit of oversimplification, but that is the gist of the idea. The average software code, however, does not allow the CPU to process a lot of instructions in parallel. Part of the reason is the way in which these instructions are inherently designed. Instructions invariably need data which is being churned out by other instructions. This dependency is an inherent hurdle to parallelising data.

Then there are complexities—huge complexities, if we are to believe software developers—in the debugging of parallel code. Tim Sweeney, of Epic Games has been quoted as saying that "Implementing a multi-threaded system requires two to three times the development and testing effort of implementing a comparable non-multithreaded system..." Tim is mostly referring to the difficulty in producing clean and parallel code. Debugging such a multi-headed beast is a monstrous task. The solution is to multi-thread manageable code—It's especially important to focus multithreading efforts on the self-contained and performance-critical subsystems that offer the most potential performance gain. You definitely don't want to execute your 150,000 lines of object-oriented gameplay logic across multiple threads—the combinatorial complexity of all of the interactions is beyond what a team can economically manage... it's vital that developers focus on self-contained systems that offer the highest effort-to-reward ratio. This effort-to-reward ratio as implemented by the latest Unreal Engine 3 is simple. As Sweeney puts it, "For multithreading optimisations, we're focusing on physics, animation updates, sound updates and content streaming. We are not attempting to multi-thread systems that are highly sequential and object-oriented, such as the gameplay".

Unreal Engine's multi-threaded Gemini engine powers games such as Gears of War

So while the actual gameplay engine might not be multi-threaded, you do see the game making use of the parallel-ability of multi-cores to offload tasks such as AI and physics calculations.

For some mundane tasks, multi-cores is the clear way ahead. Database applications, for example, scale linearly in performance with core additions and some database vendors out there are clearly pleased with the way things are heading.

But What About The Rest Of Us?

For most applications, there is currently only one thread that really needs all the power. Thus parallelisation does not bring much to the table. Some claim multi-tasking to be a much improved experience with multi-cores, and that might be so—especially, since operating systems are more multi-core aware and shuffle tasks across cores to balance things. But even multi-tasking is hostage to inherent and systemic bottlenecks. Hard drive speeds, being the biggest culprit—most applications thrash the hard-drive and there is only so much a processor can do to keep the system running smooth and fast, before it hits that particular bottleneck. Games will probably be the biggest winners due to multi-cores, with better AI, sound, physics simulations, animations and so on.

We are thus at a crossroad. The processor manufacturers want to go the multi-core route, since the other way is clearly impractical to traverse at this point. The software people though are stuck between a rock and a hard place and seem a little wary of going full-steam on the multi-lanes. Will things improve as the hardware hits its groove and compilers and other programming tools offer better solutions to the software people?

Only time will tell. ■

readersletters@thinkdigit.com